# Court Vision: Motion Planning for an Obstacle-Aware Ball-Dribbling Mobile Robot

RBE 550 Motion Planning — Project Status Update

Rudy Arsenault, Tracy Yang
Worcester Polytechnic Institute

## I. Progress Overview

At the Week 8 checkpoint, the project is on schedule against the plan submitted in the proposal. Objectives 1–3 are implemented and have been exercised in simulation; Objective 4 (redirect-the-bounce planning) and Objective 5 (evaluation) remain, as originally scheduled for Weeks 9–13. Git history shows steady, checkpoint-aligned progress: an initial working dribble by Week 5 (09-15 – 09-21), a mobile base that dribbles while turning by Week 6–7 (09-23), and a working static obstacle course with a collision-free base planner by the same week, followed by a planner re-architecture and code-quality pass this week (10-02) ahead of this submission.

One scope note from the proposal: the division of labor described there (Yang on dribble dynamics/contact control, Arsenault on obstacle mapping/ base planning) ended up reversed in practice – Arsenault built and debugged the first working dribble controller, and Yang built the obstacle course and the two-stage planner. The underlying technical split (dribble dynamics vs. planning) held; only who owns which half changed. See the Appendix for each member's specific contributions.

## II. System Implemented To Date

### A. Dribble Dynamics and Contact Model (Objective 1)

The ball is modeled in PyBullet as a sphere with a floor coefficient of restitution; between contacts it is a free projectile. At each paddle contact, outgoing vertical velocity is set by a mirror-law-style rule [1]: the commanded paddle velocity is solved so that the ball's outgoing speed is always $-v_{\text{push}}$ regardless of its incoming speed, i.e., the hand's effective restitution term cancels out of the result. A closed-form ballistic model (`BallisticModel`) predicts flight time analytically from the push speed, floor restitution, and contact/floor heights, re-anchored from the ball's real position at every contact rather than integrated open-loop, so nothing drifts between cycles. This matches the hybrid impact/flight model of [1, 2]; the first working version (09-15) had to fix an early bug where the ball spawned at rest and never had enough energy to complete its first bounce.

### B. Mobile Base Integration (Objective 2)

The robot is a kinematic chassis connected to the paddle by a fixed-length arm whose *angle* is an independent coordinate in the kinematic model, not slaved to heading via a rigid heading-plus-90° offset. It is not free in the sense of being chosen live, though: as Section II-C and Section III describe, the whole angle sequence is solved once by Objective 3's planner before the run starts, and the live loop only interpolates that precomputed sequence against the ball's progress – nothing about the arm angle is ever decided at runtime. The paddle is always set to the ball's real current position (since they must physically coincide for a contact to register), and the chassis trails it at arm's length and the commanded angle, rendered as a debug line so the arm's pose is visible during a run. Decoupling the arm angle from heading – rather than deciding anything about it live – turned out to be the key degree of freedom Objective 3's planner needed.

### C. Static Obstacle Course and Collision-Free Planning (Objective 3)

This objective is complete and now exceeds its original scope: the proposal specified 4–6 static obstacles, and the implementation generates 6–9 randomly placed and oriented crates, pillars, and walls per run. A two-stage planner, run before the simulation starts, guarantees the whole course is traversable with zero live obstacle checks during the run:

1. **Robot-body path search:** a hybrid-A* search over a state lattice of $(x, y, \text{heading})$ – heading is part of the search state, not derived afterward, so the planner can angle the chassis through a gap that is only passable at an angle. Each edge is a short forward arc; the state being searched is the robot *body's own center*, since the chassis is the part with the larger footprint that must clear every obstacle.
2. **Arm-angle sequence search:** a second, Dijkstra-style search over the already-found path that picks a collision-free arm angle (ball position relative to the chassis) at every point and verifies that every *transition* between consecutive points is collision-free, not just each point in isolation.

If a randomly generated course turns out unsolvable at either stage, generation retries with a fresh course (up to 50 attempts) rather than failing outright. The two searches' near-identical priority-queue bookkeeping was unified into one shared search routine – verified, across a ten-seed regression sweep, to produce byte-identical courses, paths, and arm-angle sequences to the pre-refactor implementation.

## III. Adaptations and Challenges

Two design pivots came directly from discovering that an initially reasonable approach could not give the guarantee the project needs.

**Reactive → proven-in-advance arm planning.** The first version of the arm-angle controller chose the chassis's offset

angle live, frame by frame, reacting to obstacles as they were encountered, with a durable-candidate lookahead and a wide safety margin around each candidate angle. This could not actually *guarantee* zero emergency corrections: it only ever verified that some angle was clear *at a point*, never that the chassis could actually get from one point's safe angle to the next's within the distance available. Three or more obstacles narrowing the safe zone from multiple sides at once could still isolate the tracked angle on an island with no reachable escape, however far ahead the lookahead searched. We replaced it with the second search stage described above, which plans the whole angle sequence up front and proves every transition collision-free before the run starts – the same philosophy already used for the robot-body path itself.

**Ball-centric → robot-body-centric planning target.** The first working planner searched the *ball's* path and derived the chassis's position as an offset from it. This was re-architected this week so the search instead plans the robot *body's* own center directly, with the ball/paddle's target path derived from it afterward by solving the same arm-offset relationship in reverse. The chassis is the part that actually has to clear every obstacle by its full footprint, so planning around it directly – rather than planning around a smaller body and hoping the larger one fits – is a better match between what is being searched and what the binding collision constraint actually is.

Separately, once the two-stage pipeline proved reliable, we increased obstacle density beyond the proposal's 4–6 range to 6–9 to stress-test the planner and produce a more demanding demonstration; Table 1 shows it still solves reliably at this density.

## IV. Preliminary Results

Table 1 summarizes ten independently-seeded random courses generated with the current 6–9-obstacle configuration, produced by `planner_benchmark.py` (`python planner_benchmark.py --latex` regenerates this exact table; `--n N` samples N fresh random seeds instead). This is a sanity-check sweep run during development, not the systematic density-vs-performance evaluation planned for Objective 5. Course generation and both planning stages are pure Python with no physics-engine dependency, so these numbers reflect planner performance in isolation.

All ten seeds solved well within the 50-attempt budget (mean 3.1 attempts, worst case 8), with planned-path overhead over the straight-line course ranging 3.3–11.7% (mean 5.3%) and total planning time under 2.3 s per course even in the worst case observed. This gives an early, favorable signal that the two-stage guarantee is cheap enough in practice to keep running at this obstacle density, not just sound in principle.

We have also added an automated MP4 recording mode (`--video`) that captures a full run end-to-end for the demonstration component of this and future deliverables.

## V. Schedule and Scope Update

Table 2 compares the original plan to actual progress. We remain aligned with the checkpoint structure from the proposal; no change to the Week 16 final deadline or the Week 14 buffer is proposed.

| Seed | Obst. | Attempts | Path (m) | Time (s) |
|---|---|---|---|---|
| 1 | 7 | 1 | 6.30 | 0.49 |
| 2 | 6 | 1 | 6.30 | 0.38 |
| 3 | 7 | 4 | 6.20 | 1.41 |
| 42 | 6 | 3 | 6.30 | 1.04 |
| 1000 | 8 | 4 | 6.30 | 1.15 |
| 999999 | 7 | 1 | 6.30 | 0.18 |
| 7 | 8 | 6 | 6.70 | 1.84 |
| 55 | 8 | 2 | 6.30 | 0.33 |
| 2024 | 6 | 8 | 6.20 | 2.24 |
| 31415 | 8 | 1 | 6.30 | 0.19 |

Table 1: Ten random seeds, 6–9 obstacles each, against a 6.00 m straight-line course. "Attempts" is regeneration attempts before a solvable course was found (max 50 allowed); "Time" is total wall-clock for course generation plus both planning stages.

| Wk | Status |
|---|---|
| 3 | *Done.* Proposal submitted; environment stood up. |
| 4–5 | *Done.* Working stationary dribble (Obj. 1), fixed an energy bug in the first version. |
| 6–7 | *Done.* Mobile base dribbles while turning (Obj. 2); static obstacle course and two-stage collision-free planner (Obj. 3), later re-architected around the robot body and increased to 6–9 obstacles. |
| 8 | **This submission.** Baseline dribble + static-obstacle navigation demonstrated; code modularized. |
| 9–10 | *Planned, as proposed.* Redirect-the-bounce planner (Obj. 4); see Remaining Work. |
| 11–12 | *Planned, as proposed.* Robustness passes; metrics/logging pipeline (Obj. 5). `ContactEvent` logging is already in place as an extension point. |
| 13 | *Planned, as proposed.* Full evaluation sweep across obstacle densities. |
| 14 | Buffer week, contingency unchanged from proposal. |
| 15–16 | Finalize results, demo video, final report/presentation. |

Table 2: Schedule status as of Week 8. No dates have moved.

## VI. Remaining Work

**Objective 4 – Redirect-the-bounce planner.** The current system already lets the *robot* weave around obstacles and lets the ball/paddle's offset angle vary to clear them, but it does not yet, as the proposal specifies, shift the ball's own landing spot in response to a blocked nominal target. Given the Section III pivot toward proving trajectories in advance rather than reacting live, our plan is to extend the existing offline search – rather than add a separate reactive layer – so that candidate landing spots become part of the same collision-free search already proving the arm-angle sequence, preserv-

ing the zero-live-correction guarantee instead of trading it away for a reactive patch.

**Objective 5 – Evaluation.** With Objective 4 in place, we will run the density sweep proposed originally (dribble-survival rate, path-length overhead, and redirect-event frequency vs. an obstacle-free baseline), using the course-generation diagnostics already logged by the current pipeline (obstacle count, attempts, path length, heading swing) as a starting point for the metrics/logging pipeline.

## Appendix: Individual Contributions

### Rudy Arsenault

Created the initial environment to get the simulation working and implemented the first working dribble test; diagnosed and fixed the energy bug that kept the ball's first bounce from reaching hand height, producing the first reliable stationary dribble (Objective 1); identified that the initial contact model produced juggling-style behavior rather than a basketball-style dribble and corrected the geometry to a two-event hybrid model — an actively controlled hand contact (paddle) above the ball and a passive, restitution-based floor bounce below it; diagnosed and fixed a corresponding object-placement overlap between the ball and paddle; verified model produced a stable periodic dribble by logging the ball's apex height and period across bounce cycles while testing.

### Tracy Yang

Built the mobile base integration (chassis + arm + paddle turning and translating while dribbling, Objective 2); designed and implemented the static obstacle course generator and the two-stage hybrid-A*/arm-angle planner (Objective 3), including the pivot from reactive to proven-in-advance arm planning and the later re-architecture to plan around the robot body directly; increased obstacle density from the proposed 4–6 to 6–9; modularized the codebase into separate concern-based files and removed duplication between the two planning searches; added automated MP4 run recording for demonstrations.

## References


[1] M. Bühler, D. E. Koditschek, and P. J. Kindlmann, "Planning and control of robotic juggling and catching tasks," *The International Journal of Robotics Research*, vol. 13, no. 2, pp. 101–118, 1994.

[2] G. Bätz, M. Sobotka, D. Wollherr, and M. Buss, "Robot basketball: Ball dribbling – a modified juggling task," in *Advances in Robotics Research*, T. Kröger and F. M. Wahl, Eds. Berlin, Heidelberg: Springer, 2009, pp. 323–334.